

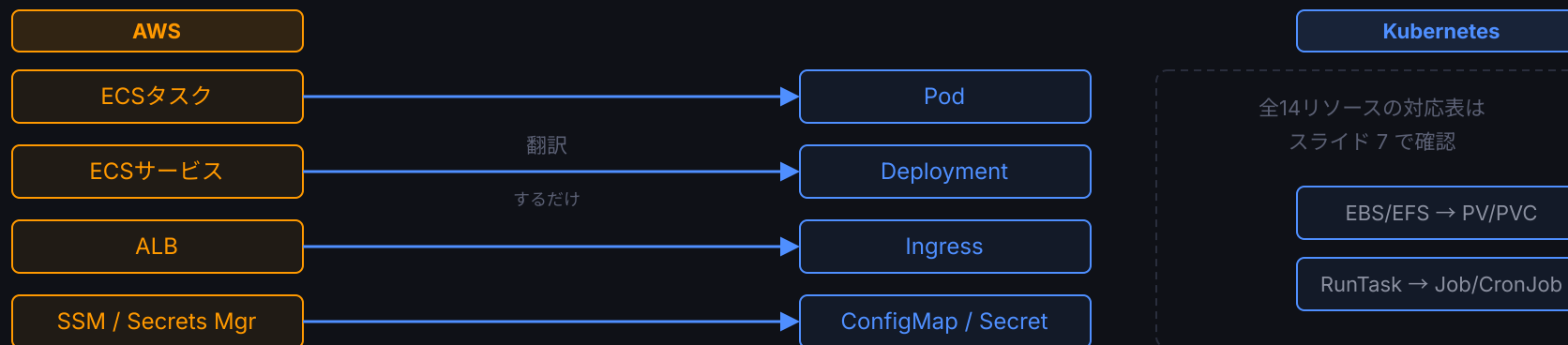
Kubernetes Foundation

AWS エンジニア向け

AWS経験者のための Kubernetes入門

ECS比較・リソース対応・マニフェストの読み方

AWSの概念と対応づけることで、KubernetesのリソースをAWSの経験から体系的に理解できる



01

AWSを抽象化した存在

KubernetesはAWS/GCP/Azureを抽象化したレイヤー。ECSタスク→Pod など知識を翻訳するだけで学習コストを大幅削減できる

02

ECSの6弱点をK8sが解決

ALB肥大化・バッチ管理・GitOps・Sidecar・環境差分・ログ集約の6課題に対してKubernetesはネイティブな解決策を持つ

03

マニフェストは3層構造

kind（何を作るか） / metadata（名前・ラベル） / spec（仕様）の3層で読める。ラベルがリソース間の接着剤になる

ECSで複雑化するとこんな壁にぶつかる

- ALBリスナールールの肥大化：マイクロサービスが増えるたびにルールが積み重なり、**100件制限**に近づく
- バッチ処理の複雑な組み合わせ：単発実行にEventBridge + Step Functions + Lambda が必要。ECSタスク単体ではリトライや完了確認が行えない
- 環境差分管理の煩雑さ：ecspressoのJSONをdev/stg/prodで別々に管理すると変更のたびに3ファイルを修正する必要がある



KubernetesがECSの6つの弱点を解決する



Job

リトライ/完了判定
マニフェストで完結

CronJob

スケジュール実行
外部サービス不要

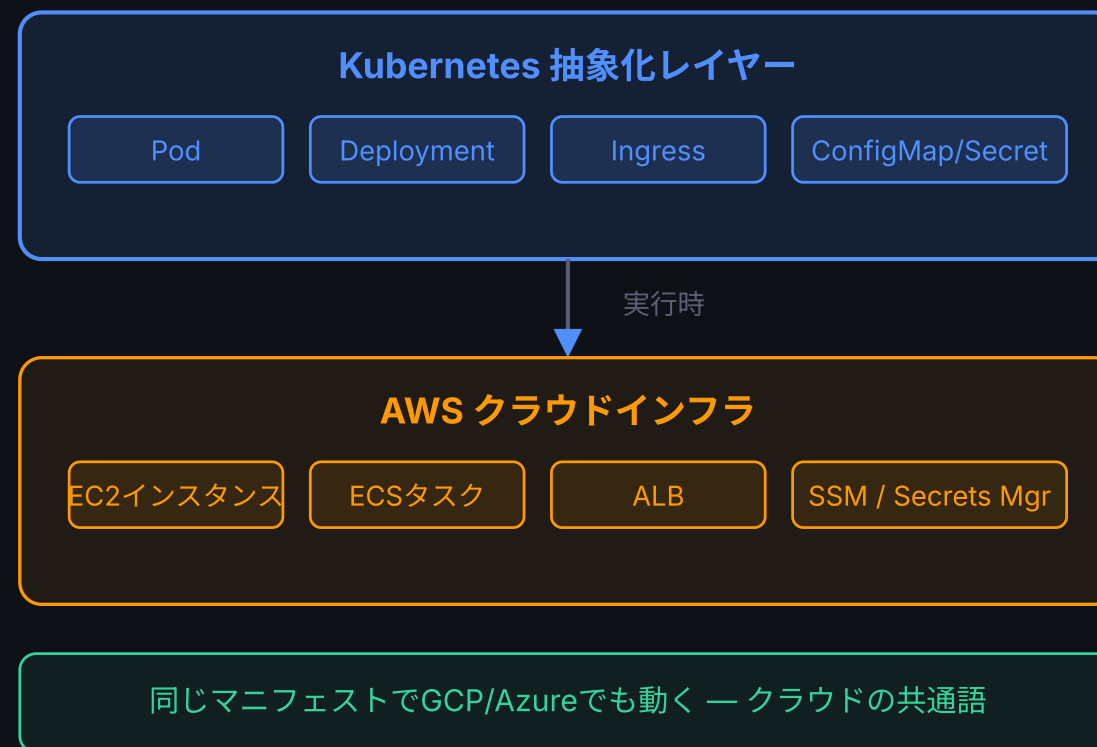
- Ingressで単一エントリーポイント：ALBは1つのまま。内部ルーティングはKubernetesが担い、チーム単位で独立管理できる
- Job / CronJobでバッチをネイティブ管理：リトライ・完了確認・スケジューリングがマニフェスト1枚で完結。外部サービス不要
- kustomizeで環境差分をパッチ管理：共通のbaseに対してenv差分だけをoverlayに記述。DRYな構成を保てる

ECS vs Kubernetes — 6つの観点で比較する

観点	ECS	KUBERNETES
ルーティング管理	ALBリスナールールがサービスごとに増殖。100件制限が現実的な課題になる	Ingressで単一エントリーポイント。チーム単位でVirtualServiceを独立管理
バッチ処理	EventBridge + Step Functions + Lambdaを組み合わせる必要がある	Job / CronJobリソースだけでリトライ・スケジューリング・完了確認が完結
環境差分管理	ecspressoのJSONを環境ごとに別ファイルで管理。変更箇所が分散する	kustomizeのbase/overlayでDRYに管理。差分だけをパッチとして記述
デプロイ方式	Push型。明示的に更新を叩かないと反映されない	Argo CDによるGitOpsでPull型。GitとクラスタをArgo CDが常に同期
ログ集約	サービスごとにCloudWatch Logsグループが分散。横断調査が困難	全PodのログがNodeに集約。Fluentdで任意の外部先に柔軟に転送可能

KubernetesはAWSを一段抽象化した存在

- クラウドに依存しない共通語：KubernetesのマニフェストはAWSでもGCPでも同じ。裏でEC2やGCEが動いていてもKubernetesの操作は変わらない
- AWSの知識が学習コストを削減：ECSタスク→Pod、ECSサービス→Deployment、ALB→Ingressと翻訳するだけで直感的に理解できる
- 抽象化の分だけ機能が増える：SchedulerやControllerによる自動配置・自動復旧・自動スケーリングがネイティブに使える



K8s ↔ AWSリソース 対応マップ

カテゴリ	Kubernetes		AWS というと
Compute	Pod	コンテナの実行単位。複数コンテナをlocalhostで共有	ECSタスク
	Node	Podが動くマシン	EC2インスタンス
Orchestration	Deployment	Podのスケール・更新・自動復旧を管理	ECSサービス + Auto Scaling
	StatefulSet	名前とストレージを固定したPodグループ	RDS（永続DB）
	ReplicaSet	Deploymentの裏でPod数を保証する仕組み	ECSサービスの必要タスク数
Networking	Service (ClusterIP)	クラスタ内部のPodへの安定したエンドポイント	ECSサービス + Cloud Map
	Ingress	L7ルーティング。パス/ホストベースで振り分け	ALB（Application Load Balancer）
	Service (NodePort)	ノードの特定ポートを外部に公開	EC2のセキュリティグループでポート公開
Config / Batch	ConfigMap	環境変数や設定ファイルをPodに注入	SSM Parameter Store / S3
	Secret	パスワード等の機密情報を安全に注入	Secrets Manager
	Job / CronJob	単発・定期バッチ処理をネイティブに管理	ECS RunTask / EventBridge Scheduler

マニフェストは3層構造で読める

```
# Deployment マニフェスト例
apiVersion: apps/v1          # ① APIグループ/バージョン
kind: Deployment              # ① 作りたいリソースの種類
metadata:
  name: slack-metrics-api     # ② リソース名(一意)
  labels:
    app: slack-metrics        # ② ラベル=リソース間の接着剤
spec:
  replicas: 3                  # ③ 起動するPod数
  selector:
    matchLabels:
      app: slack-metrics      # ③ このラベルのPodを管理
  template:
    metadata:
      labels:
        app: slack-metrics
    spec:
      containers:
        - name: api
          image: myapp:1.0.0    # ③ 使うDockerイメージ
          ports:
            - containerPort: 8080
```

① kind / apiVersion

何を作るかを宣言。Deployment・Pod・Serviceなどリソース種別をここで指定する

② metadata

nameで一意に識別。labelsはリソース間のつながりを作る「接着剤」として機能する

③ spec

「どう動かすか」の仕様。replicasでPod数、imageでコンテナを指定する

ラベルが鍵

DeploymentのselectorとPodのlabelsが一致することでリソースが関連付けられる

まとめ

01

ECSの限界でK8sが輝く

ALB肥大化・バッチ管理・GitOps・ログ集約の6課題に対してKubernetesはネイティブな解決策を持つ。マイクロサービスが複雑化したタイミングが移行の判断基準

02

AWS知識を翻訳して理解する

PodはECSタスク、DeploymentはECSサービス、IngressはALB。既存の知識を翻訳するだけでKubernetesの14リソースを直感的に理解できる

03

マニフェストは `kind/metadata/spec`

3層構造を押さえればどんなリソースも読める。ラベルがDeployment→Pod→Serviceのつながりを作る接着剤。コピペ+AIで深掘りするのが最速